

**UNIVERSITY INSTITUTE OF ENGINEERING AND
TECHNOLOGY PANJAB UNIVERSITY, SECTOR 25,
CHANDIGARH**

DIGITAL IMAGE PROCESSING LAB FILE

**CSE
DEPARTMENT
BATCH: 2023-27**



SUBMITTED BY:
Tushya Gupta
UE233106
CSE Section 2
Group 6

SUPERVISED BY:

Index

S. No.	Title	Page No.	Sign
1	Develop a program to read, display and store digital images. Understand basic image formats and pixel intensity representations.		
2	Develop a program to perform image sampling and quantization and analyze the effect of reduced resolution and grey levels on image quality.		
3	Develop a program to enhance an image using point processing techniques. Improve brightness and contrast through intensity transformations, e.g. increase brightness and contrast of a low light image.		
4	Develop a program to compute and display the histogram of a grayscale image to analyze the distribution of gray levels in the image.		
5	Develop a program to perform histogram equalization on an image.		
6	Develop a program to apply spatial filtering on an image.		
7	Develop a program edge detection in an image.		

Lab 1

Q1. Develop a program to read, display and store digital images. Understand basic image formats and pixel intensity representations.

Image format	Format specifications
PNG (Portable Network Graphics)	https://www.w3.org/TR/png/
JPEG (Joint Photographic Experts Group)	https://jpeg.org/jpeg/
AVIF (AV1 Image File Format)	https://aomediacodec.github.io/av1-avif/latest-approved.html
TIFF (Tagged Image File Format)	https://www.itu.int/itudoc/itu-t/com16/tiff-fx/docs/tiff6.pdf
GIF (Graphics Interchange Format)	https://www.w3.org/Graphics/GIF/spec-gif87.txt
WebP	https://developers.google.com/speed/webp/docs/riff_container
HEIF (High Efficiency Image File Format)	https://www.loc.gov/preservation/digital/formats/fdd/fdd000525.shtml
BitMap Image	https://learn.microsoft.com/en-us/windows/win32/gdi/bitmaps
ESP (Encapsulated PostScript)	https://www.loc.gov/preservation/digital/formats/fdd/fdd000246.shtml
RAW	https://www.loc.gov/preservation/digital/formats/fdd/fdd000241.shtml

```
#include <cstdint>
#include <cstdio>
#include <fstream>
#include <stdexcept>
#include <string>
#include <vector>
#include <zlib.h>

struct PNGChunk {
    std::string type;
    std::vector<uint8_t> data;
};

struct PNGImage {
    uint32_t width = 0;
    uint32_t height = 0;
    uint8_t bitDepth = 0;
    uint8_t colorType = 0;
    std::vector<uint8_t> pixels; // raw RGBA after decoding
};

class PNGParser {
public:
    explicit PNGParser(const std::string &filename);

    // Parse all chunks
    void parseChunks();

    // Decompress IDAT chunks to raw bytes
    std::vector<uint8_t> decompressIDAT();

    // Optionally: reconstruct pixels from decompressed scanlines
    void reconstructPixels();

    // Get parsed image info
    PNGImage getImage() const { return image; }

    void dumpChunks();

private:
    std::ifstream file;
    std::vector<PNGChunk> chunks;
    PNGImage image;

    void readSignature();
    uint32_t readUInt32();
    void readChunk();
};

PNGParser::PNGParser(const std::string &filename)
```

```
        : file(filename, std::ios::binary) {
    if (!file)
        throw std::runtime_error("Failed to open PNG file");
    readSignature();
}

void PNGParser::readSignature() {
    uint8_t sig[8];
    file.read(reinterpret_cast<char*>(sig), 8);
    const uint8_t png_sig[8] = {0x89, 'P', 'N', 'G', 0x0D, 0x0A, 0x1A, 0x0A};
    if (memcmp(sig, png_sig, 8))
        throw std::runtime_error("Not a PNG");
}

void PNGParser::dumpChunks() {
    for (auto &c : chunks) {
        printf("Type = %s\tDataL = %zu\n", c.type.c_str(), c.data.size());
    }
}

uint32_t PNGParser::readUint32() {
    uint8_t buf[4];
    file.read(reinterpret_cast<char*>(buf), 4);
    return (buf[0] << 24) | (buf[1] << 16) | (buf[2] << 8) | buf[3];
}

void PNGParser::readChunk() {
    uint32_t length = readUint32();
    char type[5] = {0};
    file.read(type, 4);

    PNGChunk chunk;
    chunk.type = type;
    chunk.data.resize(length);
    file.read(reinterpret_cast<char*>(chunk.data.data()), length);

    // skip CRC
    file.seekg(4, std::ios::cur);

    chunks.push_back(std::move(chunk));
}

void PNGParser::parseChunks() {
    while (!file.eof()) {
        readChunk(); // read next chunk

        if (chunks.empty())
            break; // safety

        if (chunks.back().type == "IEND") {
            // PNG end reached
            break;
        }
    }

    if (chunks.empty() || chunks.back().type != "IEND")
        throw std::runtime_error("Invalid PNG: missing IEND chunk");

    // Optionally, extract IHDR immediately
    for (const auto &chunk : chunks) {
```

```

        if (chunk.type == "IHDR") {
            if (chunk.data.size() != 13)
                throw std::runtime_error("Invalid IHDR size");
            image.width = (chunk.data[0] << 24) | (chunk.data[1] << 16) |
                          (chunk.data[2] << 8) | chunk.data[3];
            image.height = (chunk.data[4] << 24) | (chunk.data[5] << 16) |
                          (chunk.data[6] << 8) | chunk.data[7];
            image.bitDepth = chunk.data[8];
            image.colorType = chunk.data[9];
            break;
        }
    }
}

std::vector<uint8_t> PNGParser::decompressIDAT() {
    // Combine all IDAT chunks into a single compressed buffer
    std::vector<uint8_t> compressed;
    for (const auto &chunk : chunks) {
        if (chunk.type == "IDAT") {
            compressed.insert(compressed.end(), chunk.data.begin(), chunk.data.end());
        }
    }

    if (compressed.empty())
        throw std::runtime_error("No IDAT chunks found");

    // Estimate decompressed size: (height * (width * bytesPerPixel + 1))
    // +1 per scanline for filter byte
    int bytesPerPixel = (image.colorType == 6) ? 4 : 3; // RGBA vs RGB
    std::vector<uint8_t> decompressed(image.height *
                                      (image.width * bytesPerPixel + 1));

    z_stream strm = {};
    strm.next_in = compressed.data();
    strm.avail_in = compressed.size();
    strm.next_out = decompressed.data();
    strm.avail_out = decompressed.size();

    if (inflateInit(&strm) != Z_OK)
        throw std::runtime_error("Failed to init zlib");

    int ret = inflate(&strm, Z_FINISH);
    if (ret != Z_STREAM_END)
        throw std::runtime_error("Failed to decompress IDAT");

    inflateEnd(&strm);

    return decompressed;
}

void PNGParser::reconstructPixels() {
    std::vector<uint8_t> raw = decompressIDAT();
    int bpp = (image.colorType == 6) ? 4 : 3; // bytes per pixel
    image.pixels.resize(image.width * image.height * bpp);

    int stride = image.width * bpp;
    size_t src = 0;
    size_t dst = 0;

    for (uint32_t y = 0; y < image.height; ++y) {

```

```

uint8_t filter = raw[src++];
for (uint32_t x = 0; x < stride; ++x) {
    uint8_t val = raw[src];
    if (filter == 0) {
        // None
        image.pixels[dst++] = val;
    } else if (filter == 1) {
        // Sub
        uint8_t left = (x >= bpp) ? image.pixels[dst - bpp] : 0;
        image.pixels[dst++] = val + left;
    } else if (filter == 2) {
        // Up
        uint8_t up = (y > 0) ? image.pixels[dst - stride] : 0;
        image.pixels[dst++] = val + up;
    } else if (filter == 3) {
        // Average
        uint8_t left = (x >= bpp) ? image.pixels[dst - bpp] : 0;
        uint8_t up = (y > 0) ? image.pixels[dst - stride] : 0;
        image.pixels[dst++] = val + ((left + up) / 2);
    } else if (filter == 4) {
        // Paeth
        uint8_t left = (x >= bpp) ? image.pixels[dst - bpp] : 0;
        uint8_t up = (y > 0) ? image.pixels[dst - stride] : 0;
        uint8_t upLeft =
            (x >= bpp && y > 0) ? image.pixels[dst - stride - bpp] : 0;
        int p = left + up - upLeft;
        int pa = abs(p - left);
        int pb = abs(p - up);
        int pc = abs(p - upLeft);
        uint8_t paeth =
            (pa <= pb && pa <= pc) ? left : (pb <= pc ? up : upLeft);
        image.pixels[dst++] = val + paeth;
    }
    ++src;
}
}
}

uint32_t crc32(const uint8_t *data, size_t len) {
    return ::crc32(0, data, len); // use zlib's crc32
}

void writePNG(const std::string &filename, const PNGImage &img) {
    FILE *f = fopen(filename.c_str(), "wb");
    if (!f)
        throw std::runtime_error("Cannot open file");

    // 1. PNG signature
    uint8_t sig[8] = {0x89, 'P', 'N', 'G', 0x0D, 0x0A, 0x1A, 0x0A};
    fwrite(sig, 1, 8, f);

    // 2. IHDR chunk
    uint8_t ihdr[13];
    ihdr[0] = img.width >> 24;
    ihdr[1] = img.width >> 16;
    ihdr[2] = img.width >> 8;
    ihdr[3] = img.width;
    ihdr[4] = img.height >> 24;
    ihdr[5] = img.height >> 16;
    ihdr[6] = img.height >> 8;

```

```

ihdr[7] = img.height;
ihdr[8] = img.bitDepth;
ihdr[9] = (img.colorType == 6) ? 6 : 2; // usually 8
ihdr[10] = 0; // compression
ihdr[11] = 0; // filter
ihdr[12] = 0; // interlace

uint32_t len = htonl(13);
fwrite(&len, 4, 1, f);
fwrite("IHDR", 1, 4, f);
fwrite(ihdr, 1, 13, f);
uint8_t crcBuf[17];
memcpy(crcBuf, "IHDR", 4);
memcpy(crcBuf + 4, ihdr, 13);
uint32_t crc = crc32(crcBuf, 17);
crc = htonl(crc);
fwrite(&crc, 4, 1, f);

// 3. Build raw IDAT data with filter bytes
int bpp = (img.colorType == 6) ? 4 : 3;
std::vector<uint8_t> raw;
raw.reserve((bpp * img.width + 1) * img.height);
for (uint32_t y = 0; y < img.height; ++y) {
    raw.push_back(0); // filter type 0
    raw.insert(raw.end(), img.pixels.begin() + y * img.width * bpp,
               img.pixels.begin() + (y + 1) * img.width * bpp);
}

// 4. Compress IDAT using zlib
uLongf compSize = compressBound(raw.size());
std::vector<uint8_t> comp(compSize);
if (compress(comp.data(), &compSize, raw.data(), raw.size()) != Z_OK)
    throw std::runtime_error("Failed to compress IDAT");
comp.resize(compSize);

// 5. Write IDAT chunk
len = htonl(comp.size());
fwrite(&len, 4, 1, f);
fwrite("IDAT", 1, 4, f);
fwrite(comp.data(), 1, comp.size(), f);
std::vector<uint8_t> crcData(4 + comp.size());
memcpy(crcData.data(), "IDAT", 4);
memcpy(crcData.data() + 4, comp.data(), comp.size());
crc = crc32(crcData.data(), crcData.size());
crc = htonl(crc);
fwrite(&crc, 4, 1, f);

// 6. Write IEND chunk
len = 0;
fwrite(&len, 4, 1, f);
fwrite("IEND", 1, 4, f);
crc = crc32((const uint8_t *) "IEND", 4);
crc = htonl(crc);
fwrite(&crc, 4, 1, f);

fclose(f);
}

int main() {
    PNGParser png("image.png");

```

```
png.parseChunks();
png.reconstructPixels();
PNGImage image = png.getImage();
printf("Width = %d\nHeight = %d\nBitDepth = %d\ncolorType = %d\n",
       image.width, image.height, image.bitDepth, image.colorType);

printf("Raw pixel data size = %zu bytes\n", image.pixels.size());
png.dumpChunks();
writePNG("output.png", image);
return 0;
}
```

Lab 2

Q2. Develop a program to perform image sampling and quantization and analyze the effect of reduced resolution and grey levels on image quality.

Sampling –

```
#include "include/decode.hpp"
#include "include/sampling.hpp"
#include <stdint>
#include <cstdio>
#include <vector>

PNGImage down_sample(PNGImage &img, int k) {
    uint32_t w = img.width / k;
    uint32_t h = img.height / k;

    std::vector<uint8_t> output(w * h * 4);

    for (int y = 0; y < h; ++y) {
        for (int x = 0; x < w; ++x) {
            int sum[4] = {0, 0, 0, 0};

            for (int ky = 0; ky < k; ++ky) {
                for (int kx = 0; kx < k; ++kx) {
                    int srcX = x * k + kx;
                    int srcY = y * k + ky;

                    int idx = (srcY * img.width + srcX) * 4;

                    sum[0] += img.pixels[idx + 0];
                    sum[1] += img.pixels[idx + 1];
                    sum[2] += img.pixels[idx + 2];
                    sum[3] += img.pixels[idx + 3];
                }
            }

            int dst = (y * w + x) * 4;

            output[dst + 0] = sum[0] / (k * k);
            output[dst + 1] = sum[1] / (k * k);
            output[dst + 2] = sum[2] / (k * k);
            output[dst + 3] = sum[3] / (k * k);
        }
    }

    return PNGImage{
        .width = w,
        .height = h,
        .bitDepth = img.bitDepth,
        .colorType = img.colorType,
        .pixels = output,
    };
}
```

```

PNGImage up_sample(PNGImage &img, int k) {
    uint32_t w = img.width * k;
    uint32_t h = img.height * k;

    std::vector<uint8_t> output(w * h * 4);

    float xRatio = static_cast<float>(img.width - 1) / w;
    float yRatio = static_cast<float>(img.height - 1) / h;

    for (uint32_t y = 0; y < h; ++y) {
        for (uint32_t x = 0; x < w; ++x) {

            float gx = x * xRatio;
            float gy = y * yRatio;

            uint32_t x0 = static_cast<uint32_t>(gx);
            uint32_t y0 = static_cast<uint32_t>(gy);
            uint32_t x1 = std::min(x0 + 1, img.width - 1);
            uint32_t y1 = std::min(y0 + 1, img.height - 1);

            float dx = gx - x0;
            float dy = gy - y0;

            for (int c = 0; c < 4; ++c) {

                float p00 = img.pixels[(y0 * img.width + x0) * 4 + c];
                float p10 = img.pixels[(y0 * img.width + x1) * 4 + c];
                float p01 = img.pixels[(y1 * img.width + x0) * 4 + c];
                float p11 = img.pixels[(y1 * img.width + x1) * 4 + c];

                float value = p00 * (1 - dx) * (1 - dy) + p10 * dx * (1 - dy) +
                    p01 * (1 - dx) * dy + p11 * dx * dy;

                output[(y * w + x) * 4 + c] = static_cast<uint8_t>(value);
            }
        }
    }

    return PNGImage{
        .width = w,
        .height = h,
        .bitDepth = img.bitDepth,
        .colorType = img.colorType,
        .pixels = output,
    };
}

int main() {
    PNGParser png("images/image.png");
    png.parseChunks();
    png.reconstructPixels();

    PNGImage image = png.getImage();
    printf("==== Original Image ====\\n");
    printf("Width = %d\\nHeight = %d\\nBitDepth = %d\\ncolorType = %d\\n",
        image.width, image.height, image.bitDepth, image.colorType);
    printf("Raw pixel data size = %zu bytes\\n", image.pixels.size());

    PNGImage down_sampled_image = down_sample(image, 2);
    printf("\\n==== Down Sampled Image ====\\n");
}

```

```

printf("Width = %d\nHeight = %d\nBitDepth = %d\ncolorType = %d\n",
       down_sampled_image.width, down_sampled_image.height,
       down_sampled_image.bitDepth, down_sampled_image.colorType);
printf("Raw pixel data size = %zu bytes\n", down_sampled_image.pixels.size());
writePNG("images/down_sampled_image.png", down_sampled_image);

PNGImage up_sampled_image = up_sample(image, 2);
printf("\n==== Up Sampled Image ==== \n");
printf("Width = %d\nHeight = %d\nBitDepth = %d\ncolorType = %d\n",
       up_sampled_image.width, up_sampled_image.height,
       up_sampled_image.bitDepth, up_sampled_image.colorType);
printf("Raw pixel data size = %zu bytes\n", up_sampled_image.pixels.size());
writePNG("images/up_sampled_image.png", up_sampled_image);

return 0;
}

```

```

> ./sampling.o
==== Original Image ====
Width = 1316
Height = 816
BitDepth = 8
colorType = 6
Raw pixel data size = 4295424 bytes

==== Down Sampled Image ====
Width = 658
Height = 408
BitDepth = 8
colorType = 6
Raw pixel data size = 1073856 bytes

==== Up Sampled Image ====
Width = 2632
Height = 1632
BitDepth = 8
colorType = 6
Raw pixel data size = 17181696 bytes

```

```

void PNGParser::readChunk() {
    uint32_t length = readUInt32();
    char type[5] = {0};
    file.read(type, 4);

    PNGChunk chunk;
    chunk.type = type;
    chunk.data.resize(length);
    file.read(reinterpret_cast<char*>(chunk.data.data()), length);

    // skip CRC
    file.seek(4, std::ios::cur);

    chunks.push_back(std::move(chunk));
}

```

```

void PNGParser::readChunk() {
    uint32_t length = readUInt32();
    char type[5] = {0};
    file.read(type, 4);

    PNGChunk chunk;
    chunk.type = type;
    chunk.data.resize(length);
    file.read(reinterpret_cast<char*>(chunk.data.data()), length);

    // skip CRC
    file.seek(4, std::ios::cur);

    chunks.push_back(std::move(chunk));
}

```

up_sampled_image.png

PNG image - 491 KB

Information

[Show More](#)

Created 9 February 2026 at 10:59 AM

Modified Today, 9:52 AM

Dimensions 2632×1632

down_sampled_image.png

PNG image - 38 KB

Information

[Show More](#)

Created 9 February 2026 at 11:14 AM

Modified Today, 9:52 AM

Dimensions 658×408

Quantization –

```
#include "include/decode.hpp"

PNGImage quantize_image(PNGImage img, int levels) {
    if (levels <= 1 || levels > 256)
        return img;

    int delta = 256 / levels;

    for (size_t i = 0; i < img.pixels.size(); i += 4) {
        img.pixels[i + 0] = (img.pixels[i + 0] / delta) * delta; // R
        img.pixels[i + 1] = (img.pixels[i + 1] / delta) * delta; // G
        img.pixels[i + 2] = (img.pixels[i + 2] / delta) * delta; // B
    }

    return img;
}

int main() {
    PNGParser png("images/image.png");
    png.parseChunks();
    png.reconstructPixels();

    PNGImage image = png.getImage();
    printf("==== Original Image ==== \n");
    printf("Width = %d \n Height = %d \n BitDepth = %d \n colorType = %d \n",
        image.width, image.height, image.bitDepth, image.colorType);
    printf("Raw pixel data size = %zu bytes \n", image.pixels.size());

    PNGImage quantized_image = quantize_image(image, 2);
    printf("\n==== Down Sampled Image ==== \n");
    printf("Width = %d \n Height = %d \n BitDepth = %d \n colorType = %d \n",
        quantized_image.width, quantized_image.height,
        quantized_image.bitDepth, quantized_image.colorType);
    printf("Raw pixel data size = %zu bytes \n", quantized_image.pixels.size());
    writePNG("images/quantized_image.png", quantized_image);

    return 0;
}
```

```
void PNGParser::readChunk() {
    uint32_t length = readUInt32();
    char type[5] = {0};
    file.read(type, 4);

    PNGChunk chunk;
    chunk.type = type;
    chunk.data.resize(length);
    file.read(reinterpret_cast<char*>(chunk.data.data()), length);

    // skip CRC
    file.seekg(4, std::ios::cur);

    chunks.push_back(std::move(chunk));
}

void PNGParser::readChunk() {
    uint32_t length = readUInt32();
    char type[5] = {0};
    file.read(type, 4);

    PNGChunk chunk;
    chunk.type = type;
    chunk.data.resize(length);
    file.read(reinterpret_cast<char*>(chunk.data.data()), length);

    // skip CRC
    file.seekg(4, std::ios::cur);

    chunks.push_back(std::move(chunk));
}
```

Lab 3

Q3. Develop a program to enhance an image using point processing techniques. Improve brightness and contrast through intensity transformations, e.g. increase brightness and contrast of a low light image.

```
#include "include/decode.hpp"

void enhanceBrightnessContrast(PNGImage &img, float alpha, int beta) {
    for (size_t i = 0; i < img.pixels.size(); i += 4) {
        for (int c = 0; c < 3; c++) { // R, G, B channels
            int r = img.pixels[i + c];
            int s = static_cast<int>(alpha * r + beta);

            if (s > 255) {
                s = 255;
            }
            if (s < 0) {
                s = 0;
            }

            img.pixels[i + c] = static_cast<uint8_t>(s);
        }
        // alpha channel unchanged
    }
}

void negativeTransform(PNGImage &img) {
    for (size_t i = 0; i < img.pixels.size(); i += 4) {
        img.pixels[i] = 255 - img.pixels[i]; // R
        img.pixels[i + 1] = 255 - img.pixels[i + 1]; // G
        img.pixels[i + 2] = 255 - img.pixels[i + 2]; // B
    }
}

void powerTransform(PNGImage &img, float gamma) {
    for (size_t i = 0; i < img.pixels.size(); i += 4) {
        for (int c = 0; c < 3; c++) {

            float r = img.pixels[i + c] / 255.0f;
            float s = std::pow(r, gamma);

            int val = static_cast<int>(s * 255.0f);
            if (val > 255) {
                val = 255;
            }
            if (val < 0) {
                val = 0;
            }

            img.pixels[i + c] = static_cast<uint8_t>(val);
        }
    }
}

void contrastStretch(PNGImage &img) {
```

```
uint8_t rmin = 255;
uint8_t rmax = 0;

for (size_t i = 0; i < img.pixels.size(); i += 4) {
    for (int c = 0; c < 3; c++) {
        rmin = std::min(rmin, img.pixels[i + c]);
        rmax = std::max(rmax, img.pixels[i + c]);
    }
}

for (size_t i = 0; i < img.pixels.size(); i += 4) {
    for (int c = 0; c < 3; c++) {

        int r = img.pixels[i + c];
        int s = (r - rmin) * 255 / (rmax - rmin);

        if (s > 255) {
            s = 255;
        }
        if (s < 0) {
            s = 0;
        }
        img.pixels[i + c] = static_cast<uint8_t>(s);
    }
}

void thresholdTransform(PNGImage &img, uint8_t T) {
    for (size_t i = 0; i < img.pixels.size(); i += 4) {

        uint8_t gray = (img.pixels[i] + img.pixels[i + 1] + img.pixels[i + 2]) / 3;
        uint8_t val = (gray >= T) ? 255 : 0;

        img.pixels[i] = val;
        img.pixels[i + 1] = val;
        img.pixels[i + 2] = val;
    }
}

void piecewiselinearTransform(PNGImage &img, int r1, int s1, int r2, int s2) {
    for (size_t i = 0; i < img.pixels.size(); i += 4) {
        for (int c = 0; c < 3; c++) {

            int r = img.pixels[i + c];
            int s;

            if (r < r1)
                s = (s1 * r) / r1;
            else if (r <= r2)
                s = ((s2 - s1) * (r - r1)) / (r2 - r1) + s1;
            else
                s = ((255 - s2) * (r - r2)) / (255 - r2) + s2;

            if (s > 255) {
                s = 255;
            }
            if (s < 0) {
                s = 0;
            }
        }
    }
}
```

```
        img.pixels[i + c] = static_cast<uint8_t>(s);
    }
}

void grayLevelSlicing(PNGImage &img, int low, int high) {
    for (size_t i = 0; i < img.pixels.size(); i += 4) {

        int gray = (img.pixels[i] + img.pixels[i + 1] + img.pixels[i + 2]) / 3;
        uint8_t val;

        if (gray >= low && gray <= high)
            val = 255;
        else
            val = gray;

        img.pixels[i] = val;
        img.pixels[i + 1] = val;
        img.pixels[i + 2] = val;
    }
}

void bitPlaneSlicing(PNGImage &img, int bit) {
    for (size_t i = 0; i < img.pixels.size(); i += 4) {

        int gray = (img.pixels[i] + img.pixels[i + 1] + img.pixels[i + 2]) / 3;
        uint8_t val = ((gray >> bit) & 1) ? 255 : 0;

        img.pixels[i] = val;
        img.pixels[i + 1] = val;
        img.pixels[i + 2] = val;
    }
}

int main() {
    PNGParser parser("images/image.png");

    parser.parseChunks();
    parser.decompressIDAT();
    parser.reconstructPixels();

    PNGImage img = parser.getImage();

    // enhanceBrightnessContrast(img, 1.5f, 30);
    // writePNG("images/point_transform/brightness.png", img);

    // powerTransform(img, 0.6f); // brighten low-light image
    // writePNG("images/point_transform/gamma.png", img);

    // contrastStretch(img); // expand intensity range
    // writePNG("images/point_transform/contrast_stretch.png", img);

    // thresholdTransform(img, 120); // binary segmentation
    // writePNG("images/point_transform/threshold.png", img);

    // negativeTransform(img); // binary segmentation
    // writePNG("images/point_transform/negative.png", img);
}
```

```
// piecewiseLinearTransform(img, 70, 30, 140, 220);  
// writePNG("images/point_transform/piecewise.png", img);  
  
// grayLevelSlicing(img, 100, 150);  
// writePNG("images/point_transform/gray_slice.png", img);  
  
bitPlaneSlicing(img, 7); // extract MSB  
writePNG("images/point_transform/bitplane7.png", img);  
}
```

```
void PNGParser::readChunk() {  
    uint32_t length = readUInt32();  
    char type[5] = {0};  
    file.read(type, 4);  
  
    PNGChunk chunk;  
    chunk.type = type;  
    chunk.data.resize(length);  
    file.read(reinterpret_cast<char*>(chunk.data.data()), length);  
  
    // skip CRC  
    file.seekg(4, std::ios::cur);  
  
    chunks.push_back(std::move(chunk));  
}
```

Original Image

```

void PNGParser::readChunk() {
    uint32_t length = readUInt32();
    char type[5] = {0};
    file.read(type, 4);

    PNGChunk chunk;
    chunk.type = type;
    chunk.data.resize(length);
    file.read(reinterpret_cast<char*>(chunk.data.data()), length);

    // skip CRC
    file.seekg(4, std::ios::cur);

    chunks.push_back(std::move(chunk));
}

```

```

void PNGParser::readChunk() {
    uint32_t length = readUInt32();
    char type[5] = {0};
    file.read(type, 4);

    PNGChunk chunk;
    chunk.type = type;
    chunk.data.resize(length);
    file.read(reinterpret_cast<char*>(chunk.data.data()), length);

    // skip CRC
    file.seekg(4, std::ios::cur);

    chunks.push_back(std::move(chunk));
}

```

Brightness and contrast

```

void PNGParser::readChunk() {
    uint32_t length = readUInt32();
    char type[5] = {0};
    file.read(type, 4);

    PNGChunk chunk;
    chunk.type = type;
    chunk.data.resize(length);
    file.read(reinterpret_cast<char*>(chunk.data.data()), length);

    // skip CRC
    file.seekg(4, std::ios::cur);

    chunks.push_back(std::move(chunk));
}

```

Power transform

```

void () {
    uint32_t length = readUInt32();
    char type[5] = {0};
    file.read(type, 4);

    PNGChunk chunk;
    chunk.type = type;
    chunk.data.resize(length);
    file.read(reinterpret_cast<char*>(chunk.data.data()), length);

    // skip CRC
    file.seekg(4, std::ios::cur);

    chunks.push_back(std::move(chunk));
}

```

Contrast Stretching

```

void PNGParser::readChunk() {
    uint32_t length = readUInt32();
    char type[5] = {0};
    file.read(type, 4);

    PNGChunk chunk;
    chunk.type = type;
    chunk.data.resize(length);
    file.read(reinterpret_cast<char*>(chunk.data.data()), length);

    // skip CRC
    file.seekg(4, std::ios::cur);

    chunks.push_back(std::move(chunk));
}

```

Thresholding

```

void PNGParser::readChunk() {
    uint32_t length = readUInt32();
    char type[5] = {0};
    file.read(type, 4);

    PNGChunk chunk;
    chunk.type = type;
    chunk.data.resize(length);
    file.read(reinterpret_cast<char*>(chunk.data.data()), length);

    // skip CRC
    file.seekg(4, std::ios::cur);

    chunks.push_back(std::move(chunk));
}

```

Negative Transform

```

void () {
    uint32_t length = readUInt32();
    char type[5] = {0};
    file.read(type, 4);

    PNGChunk chunk;
    chunk.type = type;
    chunk.data.resize(length);
    file.read(reinterpret_cast<char*>(chunk.data.data()), length);

    // skip CRC
    file.seekg(4, std::ios::cur);

    chunks.push_back(std::move(chunk));
}

```

Piecewise Linear Transform

Bitplane Slicing

Lab 4

Q4. Develop a program to compute and display the histogram of a grayscale image to analyze the distribution of gray levels in the image.

```
#include "include/decode.hpp"
#include <cstdint>
#include <iostream>
#include <vector>
using namespace std;

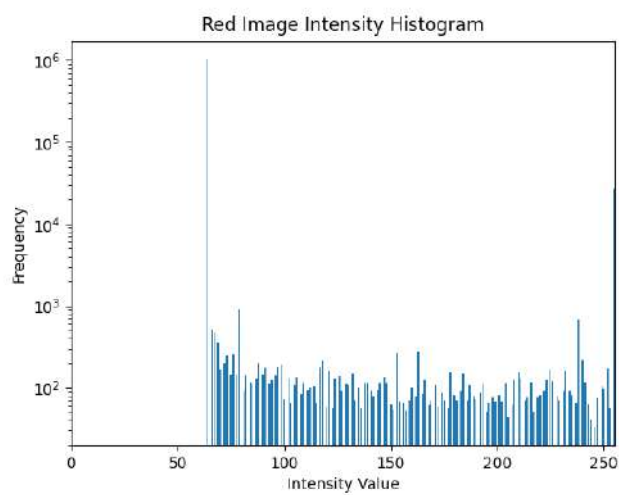
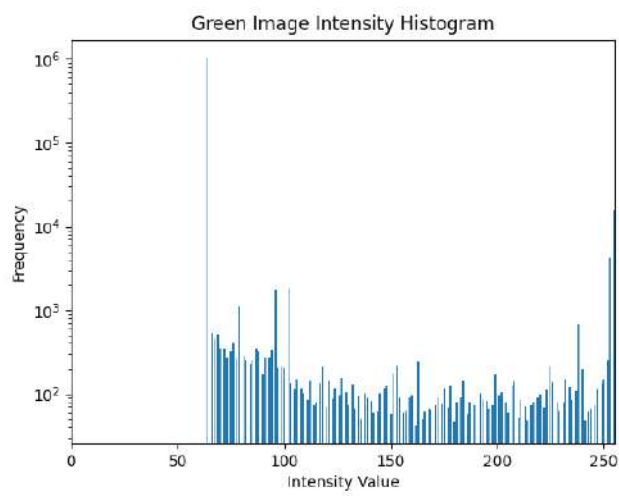
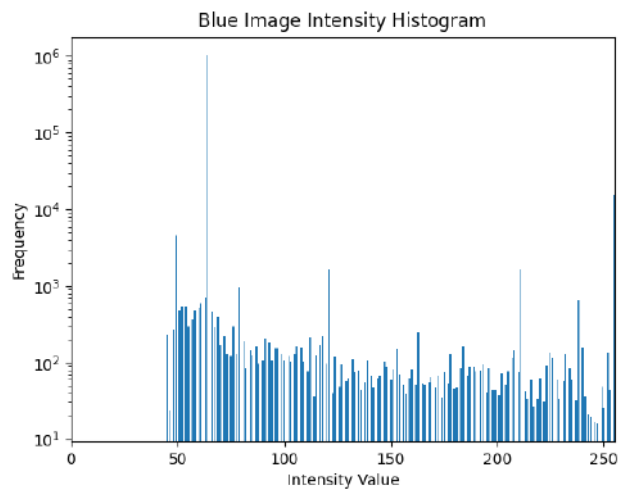
struct Histogram {
    vector<uint32_t> r;
    vector<uint32_t> g;
    vector<uint32_t> b;
};

void makeHistogram(PNGImage &img, Histogram &hist) {
    for (int i = 0; i < img.pixels.size(); i += 4) {
        uint8_t r = img.pixels[i];
        uint8_t g = img.pixels[i + 1];
        uint8_t b = img.pixels[i + 2];
        hist.r[r]++;
        hist.g[g]++;
        hist.b[b]++;
    }
}

void dumpHistogram(Histogram &hist) {
    int n = 256;
    cout << "Red\n";
    for (int i = 0; i < n; i++) {
        cout << i << ", " << hist.r[i] << "\n";
    }
    cout << "Green\n";
    for (int i = 0; i < n; i++) {
        cout << i << ", " << hist.g[i] << "\n";
    }
    cout << "Blue\n";
    for (int i = 0; i < n; i++) {
        cout << i << ", " << hist.b[i] << "\n";
    }
}

int main() {
    PNGParser parser("images/point_transform/enhanced.png");
    parser.parseChunks();
    parser.reconstructPixels();
    PNGImage img = parser.getImage();
    Histogram hist = {
        vector<uint32_t>(256, 0),
        vector<uint32_t>(256, 0),
        vector<uint32_t>(256, 0),
    };
    makeHistogram(img, hist);

    dumpHistogram(hist);
}
```



Q5. Develop a program to perform histogram equalization on an image.

```
#include "include/decode.hpp"
#include "include/histogram.hpp"
#include <stdint>
#include <iostream>
#include <unordered_map>
#include <vector>
using namespace std;

float ret_min(float a, float b) { return a < b ? a : b; }

Histogram hist_equal(Histogram &hist, uint8_t replace[][256]) {
    int n = 256;
    Histogram equalized_hist = {
        vector<uint32_t>(n, 0),
        vector<uint32_t>(n, 0),
        vector<uint32_t>(n, 0),
    };
    vector<float> pdf(n, 0);
    float sum = 0;
    for (int i = 0; i < n; i++) {
        sum += hist.r[i];
    }
    for (int i = 0; i < n; i++) {
        pdf[i] = (float)hist.r[i] / sum;
    }
    for (int i = 1; i < n; i++) {
        pdf[i] = pdf[i] + pdf[i - 1];
    }
    for (int i = 0; i < n; i++) {
        pdf[i] = ret_min(255, round(pdf[i] * (n - 1)));
    }
    // for(float e:pdf) {
    //     cout<<e<<endl;
    // }
    for (int i = 0; i < n; i++) {
        uint8_t p = static_cast<uint8_t>(pdf[i]);
        equalized_hist.r[p] += hist.r[i];
        replace[0][i] = p;
    }

    fill(begin(pdf), end(pdf), 0);
    sum = 0;
    for (int i = 0; i < n; i++) {
        sum += hist.g[i];
    }
    for (int i = 0; i < n; i++) {
        pdf[i] = (float)hist.g[i] / sum;
    }
    for (int i = 1; i < n; i++) {
        pdf[i] = pdf[i] + pdf[i - 1];
    }
}
```

```

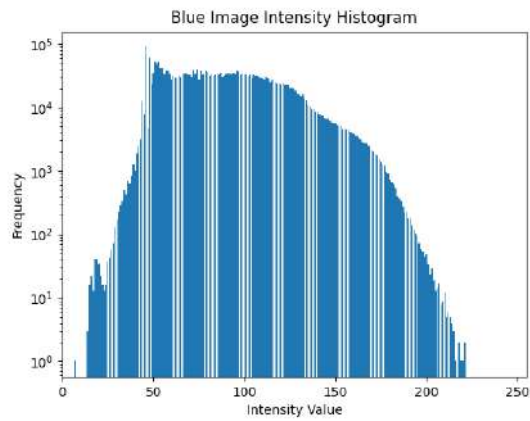
    }
    for (int i = 0; i < n; i++) {
        pdf[i] = ret_min(255, round(pdf[i] * (n - 1)));
    }
    for (int i = 0; i < n; i++) {
        uint8_t p = static_cast<uint8_t>(pdf[i]);
        equalized_hist.g[p] += hist.g[i];
        replace[1][i] = p;
    }

    fill(begin(pdf), end(pdf), 0);
    sum = 0;
    for (int i = 0; i < n; i++) {
        sum += hist.b[i];
    }
    for (int i = 0; i < n; i++) {
        pdf[i] = (float)hist.b[i] / sum;
    }
    for (int i = 1; i < n; i++) {
        pdf[i] = pdf[i] + pdf[i - 1];
    }
    for (int i = 0; i < n; i++) {
        pdf[i] = ret_min(255, round(pdf[i] * (n - 1)));
    }
    for (int i = 0; i < n; i++) {
        uint8_t p = static_cast<uint8_t>(pdf[i]);
        equalized_hist.b[p] += hist.b[i];
        replace[2][i] = p;
    }

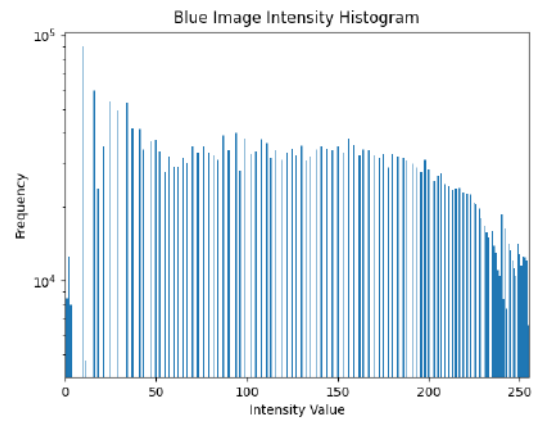
    return equalized_hist;
}

int main() {
    PNGParser parser("images/new_image.png");
    parser.parseChunks();
    parser.reconstructPixels();
    PNGImage img = parser.getImage();
    Histogram hist = {
        vector<uint32_t>(256, 0),
        vector<uint32_t>(256, 0),
        vector<uint32_t>(256, 0),
    };
    uint8_t replace[3][256] = {};
    makeHistogram(img, hist);
    Histogram equalized_hist = hist_equal(hist, replace);
    dumpHistogram(equalized_hist);
    for (int i = 0; i < img.pixels.size(); i += 4) {
        img.pixels[i] = replace[0][img.pixels[i]];
        img.pixels[i + 1] = replace[1][img.pixels[i + 1]];
        img.pixels[i + 2] = replace[2][img.pixels[i + 2]];
    }
    writePNG("images/hist_equal.png", img);
}

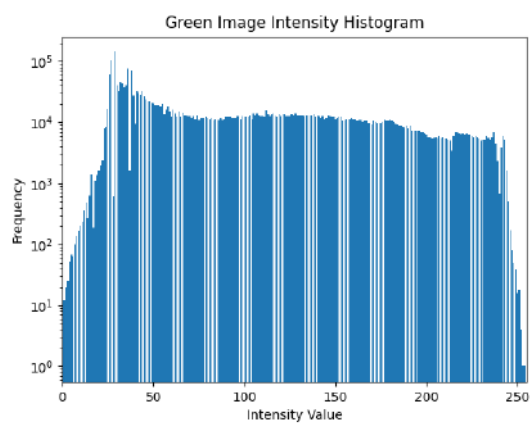
```



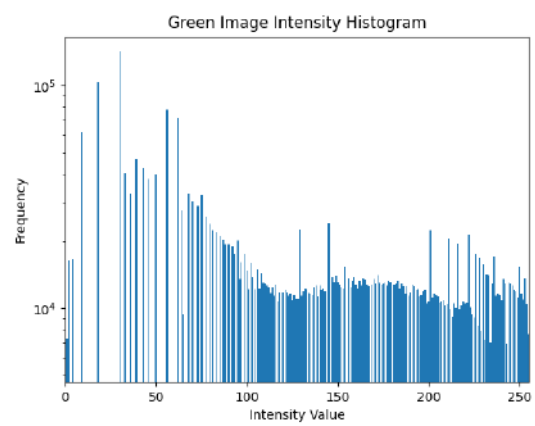
Non-Equalized



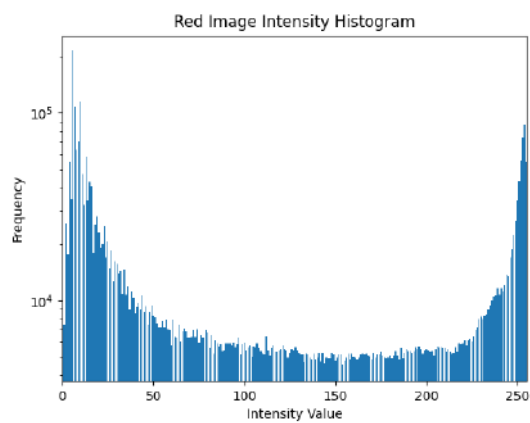
Equalized



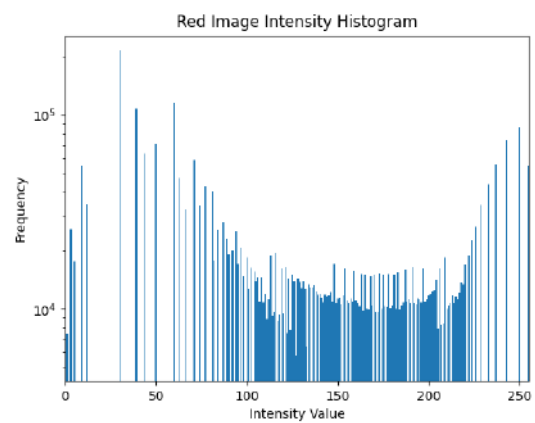
Non-Equalized



Equalized



Non-Equalized



Equalized



Non-Equalized

Equalized

Lab 5

Q6. Develop a program to apply spatial filtering on an image.

```
#include "include/decode.hpp"
#include <cmath>
#include <cstdint>
#include <iostream>
#include <vector>
using namespace std;

void get_var(vector<uint32_t> r, vector<uint32_t> g, vector<uint32_t> b) {
    double r_mean = 0, g_mean = 0, b_mean = 0;
    int n = r.size();
    for (int i = 0; i < n; i++) {
        r_mean += r[i];
        g_mean += g[i];
        b_mean += b[i];
    }
    r_mean = r_mean / n;
    g_mean = g_mean / n;
    b_mean = b_mean / n;

    double r_var = 0, g_var = 0, b_var = 0;
    for (int i = 0; i < n; i++) {
        r_var += (r[i] - r_mean) * (r[i] - r_mean);
        g_var += (g[i] - g_mean) * (g[i] - g_mean);
        b_var += (b[i] - b_mean) * (b[i] - b_mean);
    }
    r_var = r_var / n;
    g_var = g_var / n;
    b_var = b_var / n;

    cout << "R_var = " << r_var << ", G_var = " << g_var << ", B_var = " << b_var
        << endl;
}

void filter(PNGImage *img, int filter[3][3], int demo) {
    int h = img->height;
    int w = img->width;
    vector<uint32_t> r, g, b;
    for (int i = 0; i < img->pixels.size(); i += 4) {
        r.push_back(img->pixels[i]);
        g.push_back(img->pixels[i + 1]);
        b.push_back(img->pixels[i + 2]);
    }
    get_var(r, g, b);
    vector<uint32_t> r_c = r, g_c = g, b_c = b;
    int dir[9][2] = {{-1, -1}, {-1, 0}, {-1, 1}, {0, -1}, {0, 0},
                    {0, 1}, {1, -1}, {1, 0}, {1, 1}};
    for (int i = 0; i < h; i++) {
        for (int j = 0; j < w; j++) {
            double r_val = 0;
            double g_val = 0;
```

```

double b_val = 0;
for (auto d : dir) {
    int po = filter[d[0] + 1][d[1] + 1];
    int i_new = i + d[0], j_new = j + d[1];

    if (i_new < 0 || i_new >= h || j_new < 0 || j_new >= w) {
        continue;
    }

    r_val += po * r[i_new * w + j_new];
    g_val += po * g[i_new * w + j_new];
    b_val += po * b[i_new * w + j_new];
}
r_val = (1.0 / demo) * r_val;
g_val = (1.0 / demo) * g_val;
b_val = (1.0 / demo) * b_val;

r_c[i * w + j] = r_val;
g_c[i * w + j] = g_val;
b_c[i * w + j] = b_val;
}
}

get_var(r_c, g_c, b_c);

for (int i = 0, k = 0; i < img->pixels.size(); i += 4, k++) {
    img->pixels[i] = r_c[k];
    img->pixels[i + 1] = g_c[k];
    img->pixels[i + 2] = b_c[k];
}
}

void sharp_filter(PNGImage *img, int filter[3][3], int demo) {
    int h = img->height;
    int w = img->width;
    vector<double> r, g, b;
    for (int i = 0; i < img->pixels.size(); i += 4) {
        r.push_back(img->pixels[i]);
        g.push_back(img->pixels[i + 1]);
        b.push_back(img->pixels[i + 2]);
    }
    vector<double> r_c = r, g_c = g, b_c = b;
    int dir[9][2] = {{-1, -1}, {-1, 0}, {-1, 1}, {0, -1}, {0, 0},
                    {0, 1}, {1, -1}, {1, 0}, {1, 1}};
    for (int i = 0; i < h; i++) {
        for (int j = 0; j < w; j++) {
            double r_val = 0;
            double g_val = 0;
            double b_val = 0;
            for (auto d : dir) {
                int po = filter[d[0] + 1][d[1] + 1];
                int i_new = i + d[0], j_new = j + d[1];

                if (i_new < 0 || i_new >= h || j_new < 0 || j_new >= w) {
                    continue;
                }

                r_val += po * r[i_new * w + j_new];
                g_val += po * g[i_new * w + j_new];
                b_val += po * b[i_new * w + j_new];
            }
        }
    }
}

```

```

    }

    r_c[i * w + j] = r_val;
    g_c[i * w + j] = g_val;
    b_c[i * w + j] = b_val;
}
}

vector<double> r_c2 = r, g_c2 = g, b_c2 = b;
for (int i = 0; i < h; i++) {
    for (int j = 0; j < w; j++) {
        double r_val = 0;
        double g_val = 0;
        double b_val = 0;
        for (auto d : dir) {
            int po = filter[d[1] + 1][d[0] + 1];
            int i_new = i + d[0], j_new = j + d[1];

            if (i_new < 0 || i_new >= h || j_new < 0 || j_new >= w) {
                continue;
            }

            r_val += po * r[i_new * w + j_new];
            g_val += po * g[i_new * w + j_new];
            b_val += po * b[i_new * w + j_new];
        }

        r_c2[i * w + j] = r_val;
        g_c2[i * w + j] = g_val;
        b_c2[i * w + j] = b_val;
    }
}

double max_val = 0;
for (int k = 0; k < r_c.size(); k++) {
    double r_val = sqrt(r_c[k] * r_c[k] + r_c2[k] * r_c2[k]);
    double g_val = sqrt(g_c[k] * g_c[k] + g_c2[k] * g_c2[k]);
    double b_val = sqrt(b_c[k] * b_c[k] + b_c2[k] * b_c2[k]);

    max_val = max(max_val, r_val);
    max_val = max(max_val, g_val);
    max_val = max(max_val, b_val);
}

if (max_val == 0)
    max_val = 1;

for (int i = 0, k = 0; i < img->pixels.size(); i += 4, k++) {
    double r_val = sqrt(r_c[k] * r_c[k] + r_c2[k] * r_c2[k]);
    double g_val = sqrt(g_c[k] * g_c[k] + g_c2[k] * g_c2[k]);
    double b_val = sqrt(b_c[k] * b_c[k] + b_c2[k] * b_c2[k]);

    r_val = (r_val / max_val) * 255.0;
    g_val = (g_val / max_val) * 255.0;
    b_val = (b_val / max_val) * 255.0;

    img->pixels[i] = (uint8_t)r_val;
    img->pixels[i + 1] = (uint8_t)g_val;
    img->pixels[i + 2] = (uint8_t)b_val;
}

```

```

}

int main() {
    PNGParser parser("images/image.png");
    parser.parseChunks();
    parser.reconstructPixels();
    PNGImage img = parser.getImage();
    PNGImage img1, img2, img3, img4, img5;
    img1 = img2 = img3 = img4 = img5 = img;

    cout << "Mean filter\n";
    int mean_filter[3][3] = {{1, 1, 1}, {1, 1, 1}, {1, 1, 1}};
    filter(&img, mean_filter, 9);
    writePNG("images/spatial/mean.png", img);

    cout << "Weighted Avg filter\n";
    int weighted_filter[3][3] = {{1, 1, 1}, {1, 2, 1}, {1, 1, 1}};
    filter(&img1, weighted_filter, 10);
    writePNG("images/spatial/weighted.png", img1);

    cout << "Gaussian filter\n";
    int gaussian_filter[3][3] = {{1, 2, 1}, {2, 4, 2}, {1, 2, 1}};
    filter(&img2, gaussian_filter, 16);
    writePNG("images/spatial/gaussian.png", img2);

    cout << "Robert filter\n";
    int robert_filter[3][3] = {{0, 0, 0}, {0, 1, 0}, {0, 0, -1}};
    sharp_filter(&img3, robert_filter, 1);
    writePNG("images/spatial/robert.png", img3);

    cout << "Prewit filter\n";
    int prewit_filter[3][3] = {{-1, -1, -1}, {0, 0, 0}, {1, 1, 1}};
    sharp_filter(&img4, prewit_filter, 1);
    writePNG("images/spatial/prewit.png", img4);

    cout << "Sobel Avg filter\n";
    int sobel_filter[3][3] = {{-1, -2, -1}, {0, 0, 0}, {1, 2, 1}};
    sharp_filter(&img5, sobel_filter, 1);
    writePNG("images/spatial/sobel.png", img5);
}

```

```

Mean filter
R_var = 1109.07, G_var = 788.321, B_var = 726.171
R_var = 777.659, G_var = 553.41, B_var = 511.365
Weighted Avg filter
R_var = 1109.07, G_var = 788.321, B_var = 726.171
R_var = 799.327, G_var = 568.724, B_var = 525.372
Gaussian filter
R_var = 1109.07, G_var = 788.321, B_var = 726.171
R_var = 834.888, G_var = 593.932, B_var = 548.453

```

```
void PNGParser::readChunk() {
    uint32_t length = readUInt32();
    char type[5] = {0};
    file.read(type, 4);

    PNGChunk chunk;
    chunk.type = type;
    chunk.data.resize(length);
    file.read(reinterpret_cast<char*>(chunk.data.data()), length);

    // skip CRC
    file.seekg(4, std::ios::cur);

    chunks.push_back(std::move(chunk));
}
```

Mean filter

```
void PNGParser::readChunk() {
    uint32_t length = readUInt32();
    char type[5] = {0};
    file.read(type, 4);

    PNGChunk chunk;
    chunk.type = type;
    chunk.data.resize(length);
    file.read(reinterpret_cast<char*>(chunk.data.data()), length);

    // skip CRC
    file.seekg(4, std::ios::cur);

    chunks.push_back(std::move(chunk));
}
```

Weighted average filter

```
void PNGParser::readChunk() {
    uint32_t length = readUInt32();
    char type[5] = {0};
    file.read(type, 4);

    PNGChunk chunk;
    chunk.type = type;
    chunk.data.resize(length);
    file.read(reinterpret_cast<char*>(chunk.data.data()), length);

    // skip CRC
    file.seekg(4, std::ios::cur);

    chunks.push_back(std::move(chunk));
}
```

Gaussian filter

```

void PNGParser::readChunk() {
    uint32_t length = readUInt32();
    char type[5] = {0};
    file.read(type, 4);

    PNGChunk chunk;
    chunk.type = type;
    chunk.data.resize(length);
    file.read(reinterpret_cast<char*>(chunk.data.data()), length);

    // skip CRC
    file.seekg(4, std::ios::cur);

    chunks.push_back(std::move(chunk));
}

```

Robert filter

```

void PNGParser::readChunk() {
    uint32_t length = readUInt32();
    char type[5] = {0};
    file.read(type, 4);

    PNGChunk chunk;
    chunk.type = type;
    chunk.data.resize(length);
    file.read(reinterpret_cast<char*>(chunk.data.data()), length);

    // skip CRC
    file.seekg(4, std::ios::cur);

    chunks.push_back(std::move(chunk));
}

```

Prewitt filter

```

void PNGParser::readChunk() {
    uint32_t length = readUInt32();
    char type[5] = {0};
    file.read(type, 4);

    PNGChunk chunk;
    chunk.type = type;
    chunk.data.resize(length);
    file.read(reinterpret_cast<char*>(chunk.data.data()), length);

    // skip CRC
    file.seekg(4, std::ios::cur);

    chunks.push_back(std::move(chunk));
}

```

Sobel filter

Lab 6

Q7. Develop a program edge detection in an image.

```
#include "include/decode.hpp"
#include <cmath>
#include <stdint>
#include <iostream>
#include <vector>
using namespace std;

void filter(PNGImage *img, int filter[3][3], int demo) {
    int h = img->height;
    int w = img->width;
    vector<uint32_t> r, g, b;
    for (int i = 0; i < img->pixels.size(); i += 4) {
        r.push_back(img->pixels[i]);
        g.push_back(img->pixels[i + 1]);
        b.push_back(img->pixels[i + 2]);
    }
    vector<uint32_t> r_c = r, g_c = g, b_c = b;
    int dir[9][2] = {{-1, -1}, {-1, 0}, {-1, 1}, {0, -1}, {0, 0},
                    {0, 1}, {1, -1}, {1, 0}, {1, 1}};
    for (int i = 0; i < h; i++) {
        for (int j = 0; j < w; j++) {
            double r_val = 0;
            double g_val = 0;
            double b_val = 0;
            for (auto d : dir) {
                int po = filter[d[0] + 1][d[1] + 1];
                int i_new = i + d[0], j_new = j + d[1];

                if (i_new < 0 || i_new >= h || j_new < 0 || j_new >= w) {
                    continue;
                }

                r_val += po * r[i_new * w + j_new];
                g_val += po * g[i_new * w + j_new];
                b_val += po * b[i_new * w + j_new];
            }
            r_val = (1.0 / demo) * r_val;
            g_val = (1.0 / demo) * g_val;
            b_val = (1.0 / demo) * b_val;

            r_c[i * w + j] = r_val;
            g_c[i * w + j] = g_val;
            b_c[i * w + j] = b_val;
        }
    }

    for (int i = 0, k = 0; i < img->pixels.size(); i += 4, k++) {
        img->pixels[i] = r_c[k];
        img->pixels[i + 1] = g_c[k];
        img->pixels[i + 2] = b_c[k];
    }
}
```

```

    }
}

void sharp_filter(PNGImage *img, int filter[3][3], int demo) {
    int h = img->height;
    int w = img->width;
    vector<double> r, g, b;
    for (int i = 0; i < img->pixels.size(); i += 4) {
        r.push_back(img->pixels[i]);
        g.push_back(img->pixels[i + 1]);
        b.push_back(img->pixels[i + 2]);
    }
    vector<double> r_c = r, g_c = g, b_c = b;
    int dir[9][2] = {{-1, -1}, {-1, 0}, {-1, 1}, {0, -1}, {0, 0},
                    {0, 1}, {1, -1}, {1, 0}, {1, 1}};
    for (int i = 0; i < h; i++) {
        for (int j = 0; j < w; j++) {
            double r_val = 0;
            double g_val = 0;
            double b_val = 0;
            for (auto d : dir) {
                int po = filter[d[0] + 1][d[1] + 1];
                int i_new = i + d[0], j_new = j + d[1];

                if (i_new < 0 || i_new >= h || j_new < 0 || j_new >= w) {
                    continue;
                }

                r_val += po * r[i_new * w + j_new];
                g_val += po * g[i_new * w + j_new];
                b_val += po * b[i_new * w + j_new];
            }

            r_c[i * w + j] = r_val;
            g_c[i * w + j] = g_val;
            b_c[i * w + j] = b_val;
        }
    }

    vector<double> r_c2 = r, g_c2 = g, b_c2 = b;
    for (int i = 0; i < h; i++) {
        for (int j = 0; j < w; j++) {
            double r_val = 0;
            double g_val = 0;
            double b_val = 0;
            for (auto d : dir) {
                int po = filter[d[1] + 1][d[0] + 1];
                int i_new = i + d[0], j_new = j + d[1];

                if (i_new < 0 || i_new >= h || j_new < 0 || j_new >= w) {
                    continue;
                }

                r_val += po * r[i_new * w + j_new];
                g_val += po * g[i_new * w + j_new];
                b_val += po * b[i_new * w + j_new];
            }

            r_c2[i * w + j] = r_val;
            g_c2[i * w + j] = g_val;

```

```

        b_c2[i * w + j] = b_val;
    }
}

double max_val = 0;
for (int k = 0; k < r_c.size(); k++) {
    double r_val = sqrt(r_c[k] * r_c[k] + r_c2[k] * r_c2[k]);
    double g_val = sqrt(g_c[k] * g_c[k] + g_c2[k] * g_c2[k]);
    double b_val = sqrt(b_c[k] * b_c[k] + b_c2[k] * b_c2[k]);

    max_val = max(max_val, r_val);
    max_val = max(max_val, g_val);
    max_val = max(max_val, b_val);
}

if (max_val == 0)
    max_val = 1;

int thr = 50;

for (int i = 0, k = 0; i < img->pixels.size(); i += 4, k++) {
    double r_val = sqrt(r_c[k] * r_c[k] + r_c2[k] * r_c2[k]);
    double g_val = sqrt(g_c[k] * g_c[k] + g_c2[k] * g_c2[k]);
    double b_val = sqrt(b_c[k] * b_c[k] + b_c2[k] * b_c2[k]);

    r_val = (r_val / max_val) * 255.0;
    g_val = (g_val / max_val) * 255.0;
    b_val = (b_val / max_val) * 255.0;

    double mag = (r_val + g_val + b_val) / 3.0;
    uint8_t edge = (mag > thr) ? 150 : 0;

    img->pixels[i] = edge;
    img->pixels[i + 1] = edge;
    img->pixels[i + 2] = edge;
}
}

int main() {
    PNGParser parser("/Users/tushya/Downloads/self img.png");
    parser.parseChunks();
    parser.reconstructPixels();
    PNGImage img = parser.getImage();

    int gaussian_filter[3][3] = {{1, 2, 1}, {2, 4, 2}, {1, 2, 1}};
    filter(&img, gaussian_filter, 16);

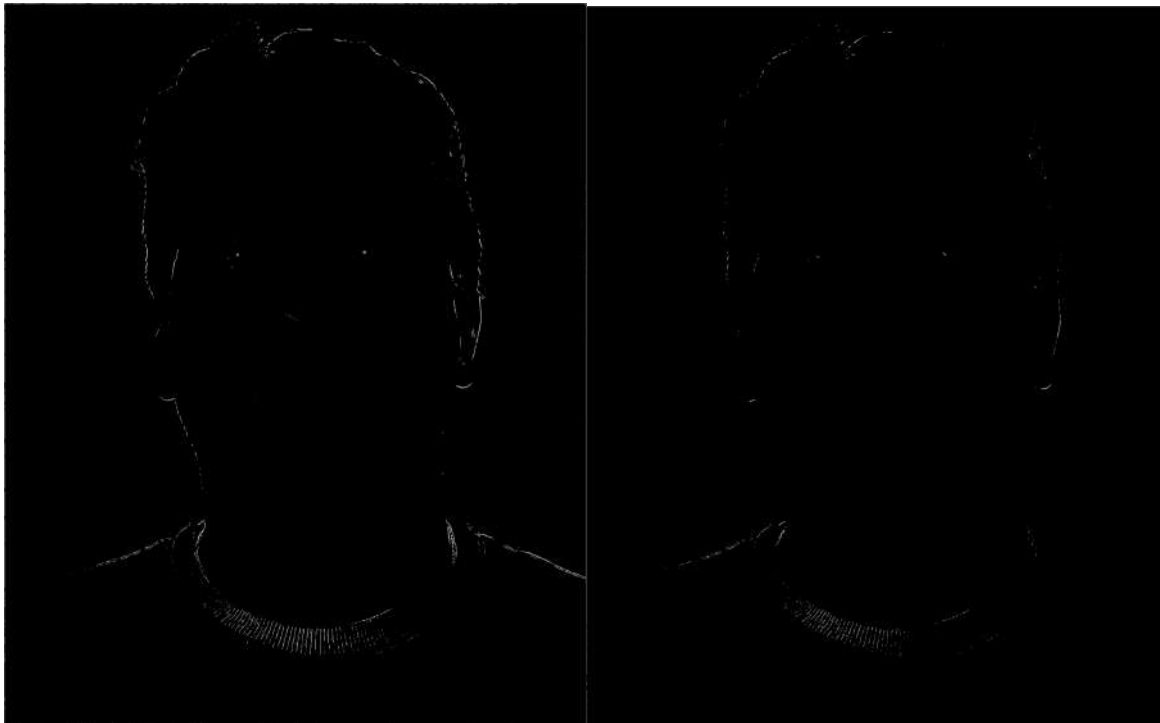
    PNGImage img1, img2, img3, img4, img5;
    img1 = img2 = img3 = img4 = img5 = img;

    cout << "Robert filter\n";
    int robert_filter[3][3] = {{0, 0, 0}, {0, 1, 0}, {0, 0, -1}};
    sharp_filter(&img3, robert_filter, 1);
    writePNG("images/spatial/robert_edge.png", img3);

    cout << "Prewit filter\n";
    int prewit_filter[3][3] = {{-1, -1, -1}, {0, 0, 0}, {1, 1, 1}};
    sharp_filter(&img4, prewit_filter, 1);
    writePNG("images/spatial/prewit_edge.png", img4);
}

```

```
cout << "Sobel Avg filter\n";  
int sobel_filter[3][3] = {{-1, -2, -1}, {0, 0, 0}, {1, 2, 1}};  
sharp_filter(&img5, sobel_filter, 1);  
writePNG("images/spatial/sobel_edge.png", img5);  
}
```



Prewit

Robert



Sobel